Journals of the Royal Statistical Society, XXXX, 1–37

doi: DOI HERE

Advance Access Publication Date: Day Month Year

Original article

Automatic Differentiation for Inference on Partially Observed Markov Process Models

K. Tan,[1] A. J. Abkemeier,[2] K. He,[2] G. Hooker[1] and E. L. Ionides[2,*]

[1]Department of Statistics, University of Pennsylvania, USA and [2]Department of Statistics, University of Michigan, USA

*Address for correspondence: Edward L. Ionides, Department of Statistics, University of Michigan, 1085 South University Avenue, Ann Arbor, MI 48109-1107, USA. ionides@umich.edu

FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

Automatic differentiation (AD) has driven recent advances in machine learning, including deep neural networks and Hamiltonian Markov Chain Monte Carlo methods. Partially observed nonlinear stochastic dynamical systems have proved resistant to AD techniques because widely used particle filter algorithms yield an estimated likelihood function that is discontinuous as a function of the model parameters. To resolve this, we create a theoretical framework that embeds two existing AD particle filter methods within a new class of algorithms. This new class permits a bias-variance tradeoff and hence a mean squared error substantially lower than the existing algorithms. This allows us to develop likelihood maximization algorithms suited to the Monte Carlo properties of the AD gradient estimate. Our algorithms require only a differentiable simulator for the latent dynamic system; by contrast, most previous approaches to AD likelihood maximization for particle filters require access to the system's transition probabilities. Numerical results indicate that a hybrid algorithm, using AD to refine a coarse solution from an iterated filtering algorithm, shows substantial improvement on current state-of-the-art methods on a challenging scientific benchmark problem.

Key words: Automatic differentiation; Hidden Markov models; Likelihood inference; Particle filtering; Sequential Monte Carlo

© The Author XXXX. Published by Oxford University Press. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

1. Introduction

Many scientific models involve highly nonlinear stochastic dynamic systems possessing significant random variation in both the process dynamics and the measurements. Commonly, the latent system is modeled as a Markov process, giving rise to a partially observed Markov process (POMP) model, also known as a hidden Markov models or a state space model. POMP models arise in fields as diverse as optimal control (Singh et al., 2023), epidemiology (He et al., 2010; Stocks et al., 2018), ecology (Knape and de Valpine, 2012) and finance (Kim and Stoffer, 2008; Bretó, 2014). Despite their ubiquity, estimation and inference for this broad class of models remains a challenging problem. We develop full-information plug-and-play inference for POMP models that takes advantage of automatic differentiation (AD) computation while controlling the Monte Carlo bias and variance of the resulting estimator. These desiderata have not previously been combined, and we discuss each component in turn.

Full-information methods are those which make statistically efficient use of data, typically via likelihood-based or Bayesian methods that avoid dependence on summary statistics. The particle filter, also known as sequential Monte Carlo, serves as the foundation for various full-information inference algorithms for POMP models, since it provides an unbiased estimate of the likelihood function (Del Moral, 2004), enabling Bayesian inference via particle Markov chain Monte Carlo (Andrieu et al., 2010) and likelihood-based inference via iterated filtering algorithms IF1 (Ionides et al., 2006) and IF2 (Ionides et al., 2015).

An inference algorithm for a POMP model is said to be plug-and-play if it does not need to evaluate the transition density of the latent Markov process, enabling an arbitrary model simulator to be plugged into the algorithm, a convenient property that facilitates scientific discovery (He et al., 2010). Some plug-and-play methods depend on the construction of low-dimensional summary statistics (Wood, 2010; Toni et al., 2009), sacrificing statistical efficiency for computational convenience. We develop computational

methods that maintain statistical efficiency and the plug-and-play property. Plug-and-play methods are sometimes called likelihood-free (Owen et al., 2015), and it may be counter-intuitive that likelihood-based inference is possible using likelihood-free methods. Basic particle filters, also known as boostrap filters, possess the plug-and-play property. Iterated particle filtering algorithms, such as IF1 and IF2, have shown that plug-and-play likelihood maximization is practical in various scientific investigations (King et al., 2008; Blake et al., 2014; Pons-Salort and Grassly, 2018; Subramanian et al., 2021; Fox et al., 2022; Drake et al., 2023). However, iterated filtering algorithms have slow convergence near the MLE (Doucet et al., 2013)

Advances in AD for gradient estimation using particle filters (Naesseth et al., 2018; Jonschkowski et al., 2018; Corenflos et al., 2021; Ścibior and Wood, 2021; Singh et al., 2023) have drawn attention to AD as a tool for inference in POMP models. Widespread adoption of these approaches have been limited by their bias (Naesseth et al., 2018; Jonschkowski et al., 2018), high variance (Poyiadjis et al., 2011; Ścibior and Wood, 2021), computational expense quadratic in the number of particles (Corenflos et al., 2021; Chen and Li, 2024), or by the loss of the plug-and-play property (Poyiadjis et al., 2011; Ścibior and Wood, 2021; Singh et al., 2023; Chen and Li, 2024). We present a new algorithm which (for reasons which will be discussed later) we call a *Differentiated Measurement Off-Parameter* (DMOP) particle filter. DMOP corresponds to the derivative of a smooth variant of the particle filter that we call MOP, though DMOP is conveniently calculated using AD without the need to actually implement the full MOP filter. MOP has the special property that, unlike most particle filters, the gradient of the filtered log-likelihood estimate provides a consistent estimate of the gradient of the log-likelihood. Particle filters have previously been adapted to provide a smooth estimate of the likelihood function (Svensson et al., 2018; Malik and Pitt, 2011), but MOP and DMOP differ from these by possessing the plug-and-play property.

DMOP trades bias against variance via a discount factor, $\alpha \in [0, 1]$. We find that DMOP-α posesses a bias-variance tradeoff that permits both low bias and low variance for

suitable choices of α strictly in $(0, 1)$, both in theory (Theorem 5) and practice (Figure 1). Theorem 1 shows that DMOP-α coincides with the biased gradient estimator of Naesseth et al. (2018) when $\alpha = 0$. In a limit as the number of particles increases, Theorem 1 also shows that DMOP-α with $\alpha = 1$ matches the high-variance gradient estimate from Poyiadjis et al. (2011) and Ścibior and Wood (2021). In this case, the variance can become prohibitively high, as discovered by these researchers and others (Arya et al., 2022). However, the variance rapidly reduces as α decreases, and so a choice of α close to one can provide a low-bias, low-variance, plug-and-play gradient estimate with computational scaling linear in the number of particles.

In practice, likelihood optimization via gradient descent with DMOP-α converges quickly within a neighborhood of the maximum but struggles to reach this neighborhood in the presence of local minima and saddle points. By contrast, iterated filtering algorithms provide a relatively fast and stable way to identify this neighborhood. We therefore build a hybrid algorithm called *Iterated Filtering with Automatic Differentiation* (IFAD) that warm-starts first-order or second-order gradient methods with a preliminary solution obtained from iterated filtering. We implement our algorithm using JAX (Bradbury et al., 2018), which also enables us to take advantage of graphical processing unit (GPU) hardware. Our methodology is provided in an open-source software library, Pypomp (Abkemeier et al., 2025).

Numerical results indicate that IFAD beats IF2 (and by the numerical results of Ionides et al. (2015), also IF1 and the Liu-West filter (Liu and West, 2001)) on a challenging problem in epidemiology, the Dhaka cholera model of King et al. (2008). IFAD also outperforms marginalized panel iterated filtering on a challenging ecological panel problem, the 38-parameter *Daphnia* model of Yang et al. (2025). Plug-and-play methods are necessary when transition densities are not available, but can have advantages even when they are. The plug-and-play property avoids difficulties arising when the full latent transition densities at different parameter values are mutually singular (Roberts

and Stramer, 2001; Karppinen et al., 2024). The plug-and-play property also facilitates the development of statistical software applicable to a broad class of POMP models.

We also present an exploratory application of DMOP-α to Bayesian computation. In Section 6, we demonstrate the DMOP-α gradient estimates within a No-U-Turn Sampler (NUTS) (Hoffman and Gelman, 2014). The numerical results provide preliminary evidence of faster parameter-space exploration than a previous application of particle Markov chain Monte Carlo (PMCMC) on the Dhaka cholera model of King et al. (2008), though we also report that adaptive choice of the PMCMC proposal can recover much of the difference.

2. Problem Setup

Consider an unobserved Markov process $\{X(t), t \geq t_0\}$, with discrete-time observations $Y_1, ..., Y_N$ realized at values $y_1^*, \ldots, y_N^*$ with times $t_1, \ldots, t_N$. The process is parameterized by an unknown parameter $\theta \in \Theta \subseteq \mathbb{R}^p$, where the state $X(t)$ takes values in the state space $\mathcal{X} \subseteq \mathbb{R}^d$, the observations Y_n take values in $\mathcal{Y}$, and we write $X_n = X(t_n)$. We suppose that the discrete-time latent process model has a density $f_{X_n|X_{n-1}}(x_n \mid x_{n-1}; \theta)$. Although we assume this density exists, plug-and-play inference algorithms do not explicitly evaluate it. Instead, they have access to a corresponding simulator, which write as $\texttt{process}_n(x_n \mid x_{n-1}; \theta)$. We set $f_{Y_n|X_n}(y_n \mid x_n; \theta)$ to be the measurement density, which can be directly evaluated. We call $f_{X_{1:n}|Y_{1:n}}(x_{1:n}|y_{1:n}^*; \theta)$ the posterior distribution of states, and $f_{X_n|Y_{1:n}}(x_n|y_{1:n}^*; \theta)$ the filtering distribution at time t_n. All joint and conditional densities are defined on a probability space $(\Omega, \Sigma, \mathbb{P})$ which is also assumed to enable construction of independent replicates and all random variables defined in our algorithms. For the algorithmic interpretation of our theory, we identify the random number seed with an element of the sample space $\omega \in \Omega$. We assume that ω can be used to construct a sequence $\{\omega_k, k \in \mathbb{Z}\}$ that comprise independent uniform random variables in the mathematical formulation and a sequence of pseudo-random numbers in the equivalent computational formulation. We require that $\texttt{process}_n$ is a function of ω_n,

though this dependence is not written explicitly. Mathematically and computationally, we can permit ω_n to be vector valued.

3. Off-Parameter Particle Filters

The *Measurement Off-Parameter with discount factor* α (MOP-α), particle filter is defined by the pseudocode in Algorithm 1. Note that this algorithm has the plug-and-play property. We suppose that the simulator is a differentiable function of θ for every fixed ω. This condition is equivalent to the *reparameterization trick* (Corenflos et al., 2021); by writing the generated random numbers as parameter-dependent transformation from a reference distribution—e.g., the quantile transform of a uniform random variable—we can differentiate their values, and hence the simulation, with respect to parameters. The discrete resampling step of the bootstrap particle filter has been the primary point of concern for the use of AD to estimate derivatives for particle filters. The discontinuity of this resampling (as a function of θ, for fixed ω) means that the derivative of the expectation is not consistently estimated by the expectation of the derivative. MOP-α is a mathematical construction designed to avoid this discontinuity. We begin with a filter that resamples particles with weights calculated at a baseline parameter value, ϕ, and then corrects these weights to target the filtering distribution at another parameter value, θ, yielding the name *measurement off-parameter*. The terminology is chosen by analogy with similar *off-policy* algorithms used for reinforcement learning in which a policy that was not implemented is evaluated by reweighting the rewards of the policy that was actually applied. If we fix ω and ϕ, the sample indices do not change with θ, but the re-weighting scheme does, allowing MOP-α to be differentiated with respect to θ. When $\theta = \phi$, we have $w_{n,j}^{P,\theta} = w_{n-1,j}^{F,\theta} = 1$ and MOP-α reduces to the standard particle filter; this is the setting we use in practice, but the construction allows us to define a derivative. One can think of MOP-α as reconstructing local smooth approximations of the discontinuous likelihood estimates produced by a particle filter for a fixed number of particles J and fixed seed ω.

These approximate the likelihood at θ in a neighborhood around ϕ, with the intent that differentiating these smooth log-likelihood reconstructions yields a score estimate.

DMOP-α (Algorithm 2) computes the fixed-seed derivative of the likelihood estimate obtained by MOP-α at $\theta = \phi$. To better understand the implementation in DMOP-α, we review some background for AD. Standard AD consists of two evaluation passes: a forward pass in which the function is evaluated and a graph built of the evaluation process, followed by a backward pass where the derivative is evaluated by recursive use of the chain rule on the computation graph (Griewank and Walther, 2008). The two evaluation passes in MOP-α at ϕ and θ correspond to the two AD evaluation passes in DMOP-α. Differentiation of MOP-α with respect to θ requires a preliminary evaluation at ϕ and subsequent differentiation with respect to θ for the second pass which we obtain through AD. In practice, however, we only work in the $\theta = \phi$ setting in which case MOP-α corresponds to the standard particle filter and we only need to make one pass. In order to apply AD in this setting without using two passes, we must both instantiate the weighting scheme (even though the weights are always 1) and distinguish which components of the weights correspond to which pass. This can be achieved using the *stop-gradient* capability that appears in Algorithm 2 where its application in the weight correction step indicates that AD should apply the chain rule to the numerator but not the denominator of the correction; note that this construction yields weights which are always 1, but which also have non-zero derivatives.

MOP-α has similarities with the algorithm of Svensson et al. (2018), in which particles from a previous run at ϕ are reweighted to evaluate the likelihood at any sufficiently nearby parameter θ for 0-th order optimization. However, their method is not plug-and-play because they use transition densities to reweight particles, and the variance of their log-likelihood estimate has an exponential dependence in the horizon, N, when $\theta \neq \phi$. Our approach bypasses these issues: the first since the reparameterization trick allows us to consider particle paths dependent on θ, and the second as using AD allows us to only evaluate the likelihood and score estimates when $\theta = \phi$.

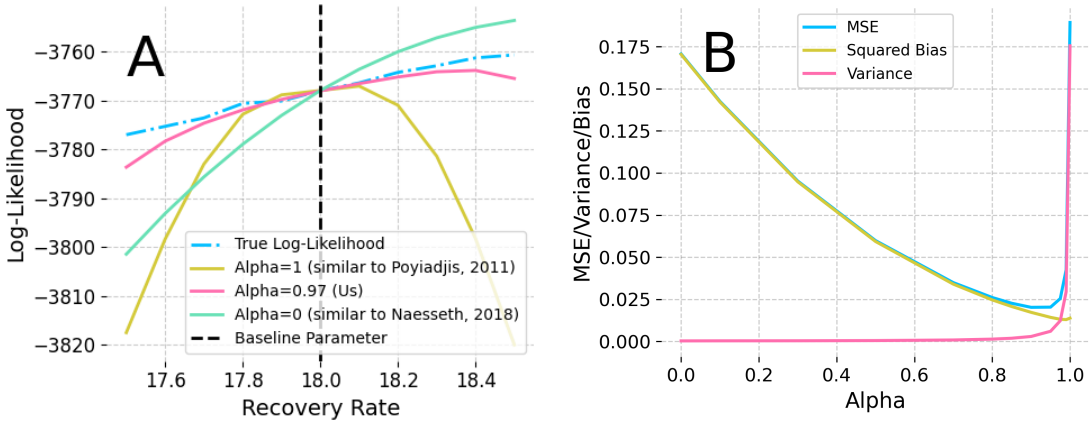


Fig. 1. A: Non-differentiable likelihood estimates and smooth off-parameter approximations that hold locally around the baseline parameter ϕ. **B:** Illustration of the bias-variance tradeoff induced by the discounting hyperparameter α. We display the MSE of score estimates for the recovery rate at the MLE for the Dhaka cholera model of King et al. (2008). In this example, intermediate values of α improve the likelihood approximation and the score estimates.

3.1. Some algorithmic details

The resampling indices, which we write as $k_j \sim \text{Categorical}(g^{\phi}_{n,1}, ..., g^{\phi}_{n,J})$, are a function of ϕ for any value of θ. For the categorical distribution, we use systematic resampling (Arulampalam et al., 2002; King et al., 2016) which usually outperforms multinomial resampling. The finite-sample concentration result used for Theorem 2, however, is stated for multinomial resampling; it does not assert the same guarantee for the systematic-resampling implementation used in our numerical work.

In Algorithm 1, the conditional likelihoods are reweighted by a correction factor $w^{P,\theta}_{n,j}$ that accumulates over filtering steps to account for the resampling under ϕ. Writing $g^{\theta}_{n,j} = f_{Y_n|X_n}(y^*_n|X^{P,\theta}_{n,j};\theta)$ for the measurement density and $L^{\phi}_n = \frac{1}{J}\sum_{m=1}^{J} g^{\phi}_{n,m}$ for the conditional likelihood estimate under ϕ, we obtain two estimates of the conditional

Algorithm 1 MOP-α

Input: Number of particles J, timesteps N, measurement model $f_{Y_n|X_n}(y_n^*|x_n,\theta)$, simulator $\texttt{process}_n(\cdot|x_n;\theta)$, evaluation parameter θ, baseline parameter ϕ, seed ω.

First pass: Set $\theta=\phi$ and fix ω, yielding $X_{n,j}^{P,\phi}$, $X_{n,j}^{F,\phi}$, $g_{n,j}^{\phi}$.

Second pass: Fix ω, and filter at $\theta\neq\phi$:

Initialize particles $X_{0,j}^{F,\theta}\sim f_{X_0}(\cdot\,;\theta)$, weights $w_{0,j}^{F,\theta}=1$.

For $n=1,...,N$:

Accumulate discounted weights, $w_{n,j}^{P,\theta}=\big(w_{n-1,j}^{F,\theta}\big)^{\alpha}$.

Simulate process model, $X_{n,j}^{P,\theta}\sim\texttt{process}_n\big(\,\cdot\,|X_{n-1,j}^{F,\theta};\theta\big)$.

Measurement density, $g_{n,j}^{\theta}=f_{Y_n|X_n}(y_n^*|X_{n,j}^{P,\theta};\theta)$.

Compute $L_n^{B,\theta,\alpha}=\sum_{j=1}^{J}g_{n,j}^{\theta}\,w_{n,j}^{P,\theta}\,/\sum_{j=1}^{J}w_{n,j}^{P,\theta}$.

Conditional likelihood under ϕ, $L_n^{\phi}=\frac{1}{J}\sum_{m=1}^{J}g_{n,m}^{\phi}$.

Select resampling indices $k_{1:J}$ with $\mathbb{P}\big(k_j=m\big)\propto g_{n,m}^{\phi}$.

Obtain resampled particles $X_{n,j}^{F,\theta}=X_{n,k_j}^{P,\theta}$.

Calculate corrected weights $w_{n,j}^{F,\theta}=w_{n,k_j}^{P,\theta}\,g_{n,k_j}^{\theta}\,/\,g_{n,k_j}^{\phi}$.

Compute $L_n^{A,\theta,\alpha}=L_n^{\phi}\cdot\sum_{j=1}^{J}w_{n,j}^{F,\theta}\,/\sum_{j=1}^{J}w_{n,j}^{P,\theta}$.

Return: likelihood estimate $\hat{\mathcal{L}}^A(\theta)=\prod_{n=1}^{N}L_n^{A,\theta,\alpha}$ or $\hat{\mathcal{L}}^B(\theta)=\prod_{n=1}^{N}L_n^{B,\theta,\alpha}$, log-likelihood $\hat{\ell}^A(\theta)=\log\big(\hat{\mathcal{L}}^A(\theta)\big)$ or $\hat{\ell}^B(\theta)=\log\big(\hat{\mathcal{L}}^B(\theta)\big)$, and filtering distributions $\{(X_{n,j}^{F,\theta},w_{n,j}^{F,\theta})\}_{n,j=1}^{N,J}$.

likelihood under θ,

$$L_n^{B,\theta,\alpha}=\frac{\sum_{j=1}^{J}g_{n,j}^{\theta}\,w_{n,j}^{P,\theta}}{\sum_{j=1}^{J}w_{n,j}^{P,\theta}},\qquad L_n^{A,\theta,\alpha}=L_n^{\phi}\cdot\frac{\sum_{j=1}^{J}w_{n,j}^{F,\theta}}{\sum_{j=1}^{J}w_{n,j}^{P,\theta}},\tag{1}$$

where we recursively correct the weights for each particle for the cumulative error incurred by the off-parameter resampling:

$$w_{n,j}^{P,\theta}=(w_{n-1,j}^{F,\theta})^{\alpha},\quad w_{n,j}^{F,\theta}=w_{n,k_j}^{P,\theta}\,g_{n,k_j}^{\theta}\,/g_{n,k_j}^{\phi},\quad w_{0,j}^{F,\theta}=1.\tag{2}$$

Algorithm 2 DMOP-α

Input: Number of particles J, timesteps N, measurement model $f_{Y_n|X_n}(y_n^*|x_n,\theta)$, simulator $\texttt{process}_n(\cdot|x_n;\theta)$, parameter θ.

Forward pass: AD constructs a computation graph for the likelihood estimate, $\hat{\mathcal{L}}^A(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\hat{\mathcal{L}}^B(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$.

Backward pass: AD evaluates the gradient estimate, $\hat{\mathcal{D}}^A(\theta)$ or $\hat{\mathcal{D}}^B(\theta)$.

Initialize particles $X_{0,j}^{F,\theta} \sim f_{X_0}(\cdot\,;\theta)$, weights $w_{0,j}^{F,\theta} = 1$.

For $n = 1, ..., N$:

Accumulate discounted weights, $w_{n,j}^{P,\theta} = \big(w_{n-1,j}^{F,\theta}\big)^\alpha$.

Simulate process model, $X_{n,j}^{P,\theta} \sim \texttt{process}_n\big(\,\cdot\,|X_{n-1,j}^{F,\theta};\theta\big)$.

Measurement density, $g_{n,j}^\theta = f_{Y_n|X_n}(y_n^*|X_{n,j}^{P,\theta};\theta)$.

Compute $L_n^{B,\theta,\alpha} = \sum_{j=1}^J g_{n,j}^\theta\, w_{n,j}^{P,\theta} \,/\, \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Select resampling indices $k_{1:J}$ with $\mathbb{P}\big(k_j = m\big) \propto g_{n,m}^\theta$.

Obtain resampled particles $X_{n,j}^{F,\theta} = X_{n,k_j}^{P,\theta}$.

Calculate corrected weights $w_{n,j}^{F,\theta} = w_{n,k_j}^{P,\theta}\, g_{n,k_j}^\theta \,/\, \texttt{stop_gradient}(g_{n,k_j}^\theta)$.

Compute $L_n^{A,\theta,\alpha} = \texttt{stop_gradient}\big(L_n^{B,\theta,\alpha}\big) \sum_{j=1}^J w_{n,j}^{F,\theta} \,/\, \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Return: log-likelihood estimate $\hat{\ell}^A(\theta) = \log\big(\hat{\mathcal{L}}^A(\theta)\big) = \sum_{n=1}^N \log L_n^{A,\theta,\alpha}$ or $\hat{\ell}^B(\theta) = \log\big(\hat{\mathcal{L}}^B(\theta)\big) = \sum_{n=1}^N \log L_n^{B,\theta,\alpha}$, and gradient estimate $\hat{\mathcal{D}}^A(\theta)$ or $\hat{\mathcal{D}}^B(\theta)$.

The before-resampling estimate $L_n^{B,\theta,\alpha}$ is preferable in practice, as it has slightly lower variance than the after-resampling estimate $L_n^{A,\theta,\alpha}$, but the latter is useful in deriving properties of the MOP-α gradient estimate such as that in Theorem 1.

3.2. Discounted Weights and a Bias-Variance Tradeoff

Heuristically, α controls the exponentially decaying memory of MOP and DMOP. At $\theta = \phi$, the weights equal one, so α does not affect the function evaluation in the first pass of DMOP. It does affect the gradients evaluated in the second pass. If $\alpha = 1$, the gradient of the weights, $w_{n,j}^{P,\theta}$ and $w_{n,j}^{F,\theta}$, accumulates as n increases. This leads to numerical instability and high variance unless N is small. For $\alpha < 1$, the weights from

previous timesteps are discounted as in Equation (2), allowing for lower variance at the expense of bias in the gradient estimate.

When $\alpha = 1$, MOP-1 maintains complete memory of each particle's ancestral trajectory. Theorem 1 shows that DMOP-1 has a similar form to the consistent but high-variance score estimate in the first algorithm of Poyiadjis et al. (2011). Unlike Poyiadjis et al. (2011), our algorithm has the plug-and-play property; in the case $\alpha = 1$, our algorithm replaces differentiation of the latent transition density in Poyiadjis et al. (2011) by pathwise differentation of the simulator.

When $\alpha = 0$, MOP-0 considers only single-step transition dynamics, and the before-sampling variant of DMOP-0 recovers the low-variance but asymptotically biased score estimate of Naesseth et al. (2018). Ścibior and Wood (2021) showed that this matches the gradient of a vanilla particle filter when resampling terms are dropped.

Theorem 1 (DMOP-0 and DMOP-1 Functional Forms)**.** *Write $\nabla_\theta \hat{\ell}^\alpha(\theta)$ for the gradient estimate yielded by DMOP-α and use the after-resampling conditional likelihood estimate so that $\hat{\ell}(\theta) = \log \hat{\mathcal{L}}(\theta) = \log \prod_{n=1}^N L_n^{A,\theta,\alpha}$. When $\alpha = 0$,*

$$\nabla_\theta \hat{\ell}^0(\theta) = \frac{1}{J} \sum_{n=1}^{N} \sum_{j=1}^{J} \nabla_\theta \log f_{Y_n|X_n}\left(y_n^* | X_{n,j}^{F,\theta}; \theta\right),$$

where the derivative with respect to θ on the right hand side propagates the derivative of the particle locations, via the chain rule, for fixed ω. When $\alpha = 1$,

$$\nabla_\theta \hat{\ell}^1(\theta) = \frac{1}{J} \sum_{j=1}^{J} \nabla_\theta \log f_{Y_{1:N}|X_{1:N}}\left(y_{1:N}^* | X_{1:N,j}^{A,F,\theta}; \theta\right).$$

Here, $X_{1:N,j}^{A,F,\theta}$ is the ancestral trajectory of $X_{N,j}^{F,\theta}$ at times $t_1, \ldots, t_N$.

We defer the proof to the supplement (Section S1). The argument relies on a useful decomposition of the after-resampling estimate $L_n^{A,\theta,\alpha}$, repeated applications of the log-derivative identity that $\nabla_x \log(f(x)) = (\nabla_x f(x))/f(x)$, and noting $\theta = \phi$ implies that $w_{n,j}^{P,\theta}$ evaluates to 1.

The result in Theorem 1 provides mathematical intuition for why Ścibior and Wood (2021)'s use of the stop-gradient procedure in the particle filter recovers the estimate of Poyiadjis et al. (2011). When $\theta = \phi$, the correction $g_{n,k_j}^{\theta}/g_{n,k_j}^{\phi}$ is equivalent to $g_{n,k_j}^{\theta}/\text{stop_gradient}(g_{n,k_j}^{\theta})$, and is implemented in this way in practice. As such, the use of the stop-gradient in Ścibior and Wood (2021) can be seen as the MOP-α weight correction for $\alpha = 1$, and $\theta = \phi$.

4. From Gradients to Maximum Likelihood Estimation

To design an effective procedure for likelihood maximization using DMOP-α, we carry out iterated filtering then automatic differentiation (IFAD). A basic IFAD approach is presented in Algorithm 3. The second stage uses a symmetric positive-definite curvature matrix B_m whose eigenvalues are kept in a fixed interval $[c, C]$. A raw particle Hessian or a quasi-Newton matrix may be spectrally clipped to this interval; the choice $B_m = I_p$ gives stochastic gradient descent (SGD). Algorithm 3 is convenient for theoretical analysis, but two further modifications are helpful in practice. Our best numerical results were obtained by putting the gradient estimate into the Adam optimizer (Kingma and Ba, 2015) together with learning rate decay. Numerical results are provided in Section 5.

The result below analyzes the specialization of Algorithm 3 that uses $\alpha = 1$, the after-resampling likelihood estimate, and multinomial resampling. These restrictions are needed for the stated finite-sample probability bound.

The convergence of IF2 (and so the first stage of IFAD) to a neighborhood of the MLE happens rapidly in practice. However, even when applying much larger amounts of Monte Carlo effort, IF2 fails to get much closer than 2 log-likelihood units of the maximum on this problem (Figure 3). For this example, IF2, without assistance from AD, struggles to

Algorithm 3 IFAD

Input: Number of particles J, timesteps N, IF2 cooling schedule a_m, AD step size η, MOP-α discount parameter α, spectral bounds $0 < c \leq C$, gradient tolerance ϵ, stopping multiplier σ, maximum AD iterations M, and initial value θ_0.

Run the initial IF2 search under cooling schedule a_m to obtain $\{\Theta_j, j = 1, ..., J\}$, and set $\theta_0 := J^{-1} \sum_{j=1}^{J} \Theta_j$.

For $m = 0, \ldots, M-1$:

Using fresh random numbers, run Algorithm 1 to obtain $\hat{\ell}(\theta_m)$ and set $g_m := \nabla_\theta(-\hat{\ell}(\theta_m))$.

If $\|g_m\|_2 \leq \sigma\epsilon$, **return** $\hat{\theta} := \theta_m$.

Choose a symmetric raw curvature estimate $\widetilde{B}_m$ and clip its eigenvalues to $[c, C]$ to obtain B_m.

Update $\theta_{m+1} := \theta_m - \eta B_m^{-1} g_m$.

Return $\hat{\theta} := \theta_M$.

reach the inferentially critical region within one log-likelihood unit of the maximum, even when additional computationally intensive techniques are used, such as repeated pruning and profiling,

For a fixed optimization budget, the AD stage has a high-probability linear convergence rate up to a neighborhood of the MLE that shrinks as the particle gradient accuracy improves, under the usual smoothness and strong convexity assumptions, as shown below.

Theorem 2 (Linear Convergence of IFAD)**.** *Condition on the IF2 output θ_0 and write $f = -\ell$. Suppose f is γ-strongly convex and Γ-smooth on a convex region containing the second-stage trajectory, with unique minimizer θ^*, and that (A3) and (A5) hold uniformly there. Consider the $\alpha = 1$, after-resampling, multinomial specialization of Algorithm 3, using fresh particle randomness at each iteration and curvature matrices $cI_p \preceq B_m \preceq CI_p$.*

Fix $M \geq 1$, $\delta \in (0, 1)$, $\epsilon > 0$, and $\beta \in (0, 1)$, and choose $0 < \eta \leq c(1-\beta)/\Gamma$ and $\sigma \geq 2C/[c(1-\beta)]$. For J sufficiently large, as quantified in Section S2, with conditional

probability at least $1-\delta$ *every nonstopping iteration up to* M *satisfies*

$$\ell(\theta^*) - \ell(\theta_{m+1}) \le q\{\ell(\theta^*) - \ell(\theta_m)\}, \qquad q = 1 - \frac{2\eta\beta\gamma}{C}\left(\frac{\sigma}{\sigma+1}\right)^2 \in (1/2, 1).$$

Thus, if no stop occurs, the final likelihood gap is at most $q^M\{\ell(\theta^*) - \ell(\theta_0)\}$*; if the algorithm stops at* θ_τ*, then* $\|\nabla\ell(\theta_\tau)\|_2 \le (\sigma+1)\epsilon$ *and* $\ell(\theta^*) - \ell(\theta_\tau) \le (\sigma+1)^2\epsilon^2/(2\gamma)$.

The proof of Theorem 2 is a finite-horizon inexact-gradient argument. It retains the two-sided spectral control and the per-iteration probability accounting that are essential in the related subsampled-Newton analysis of Roosta-Khorasani and Mahoney (2016). Details are deferred to the supplement (Section S2). This result shows that, over any prespecified finite optimization budget, particle gradient estimates can retain a linear rate until their Monte Carlo accuracy becomes the limiting factor. The theorem is stated for DMOP-1: an analogous result for $\alpha < 1$ additionally requires a high-probability bound on the discount bias relative to the exact score and is not implied merely by taking α close to one.

We therefore see that the second stage of IFAD converges linearly toward a Monte Carlo neighborhood of the MLE if (1) the log-likelihood surface is well-conditioned in a neighborhood of the MLE and (2) the first stage of IFAD successfully reaches a basin in which the theorem's assumptions hold. Within a neighborhood of the MLE that grows as more data become available (Ning et al., 2021), local quadratic approximations such as local asymptotic normality apply under general conditions (Le Cam and Yang, 2000). This suggests that the linear convergence may occur often in practice.

5. Application to a Cholera Transmission Model

The cholera transmission model that King et al. (2008) developed for Dhaka, Bangladesh, has been used to benchmark the performance of various POMP inference methods (Ionides et al., 2015; Fasiolo et al., 2016; Wycoff et al., 2026), and we employ it here for the same purpose. This model categorizes individuals in a population as susceptible, $S(t)$, infected,

$I(t)$, and recovered, $R(t)$ and so is called an SIRS compartmental model. In this case, the compartment $R(t)$ is further subdivided into three recovered compartments $R^1(t)$, $R^2(t)$, $R^3(t)$ denoting varying degrees of cholera immunity. We write $P(t)$ for the total population, and M_n for the cholera deaths in each month. As in King et al. (2008), the transition dynamics follow a stochastic differential equation (SDE):

$$\begin{aligned} dS &= \big(k\epsilon R^k + \delta(P-S) - \lambda(t)S\big)\,dt + dP - (\sigma SI/P)\,dB, \\ dI &= \big(\lambda(t)S - (m+\delta+\gamma)I\big)\,dt + (\sigma SI/P)\,dB, \\ dR^1 &= \big(\gamma I - (k\epsilon+\delta)R^1\big)\,dt, \quad \dots \\ dR^k &= \big(k\epsilon R^{k-1} - (k\epsilon+\delta)R^k\big)\,dt, \end{aligned}$$

with Brownian motion $B(t)$, cholera death rate m, recovery rate γ, mean immunity duration $1/\epsilon$, standard deviation of the force of infection σ, and population death rate $\delta = 0.02$. The force of infection, λ_t, is modeled by splines $(s_j)_{j=1}^6$

$$\lambda_t = \exp\!\left(\beta_{\text{trend}}(t-t_0) + \sum_{j=1}^{6}\beta_j s_j(t)\right)\frac{I}{P} + \exp\!\left(\sum_{j=1}^{6}\omega_j s_j(t)\right),$$

where the coefficients $(\beta_j)_{j=1}^6$ model seasonality in the force of infection, β_{trend} models the trend in the force of infection, and the ω_j represent seasonality of a non-human environmental reservoir of disease. The measurement model for observed monthly cholera deaths is $Y_n \sim \mathcal{N}(M_n, \tau^2 M_n^2)$, where $M_n = m\int_{t_{n-1}}^{t_n} I(s)\,ds$ is the modeled cholera deaths in that month.

We numerically solve this SDE using Euler's method with a time step of 1/20 months, matching (King et al., 2008). This example is a partially observed hypo-elliptical SDE, and methodology specific to this model class has been previously studied by Ditlevsen and Samson (2019) (henceforth, DS19). Supplementary Section S11 compares IFAD with an adaptation of DS19 and the differentiable particle filter of Corenflos et al. (2021)

(CTDD21), including the use of a warm start using IF2. Section S12 expands the comparison to five models of varying complexity. IFAD is found to be advantageous on the more complex situations such as this Dhaka model. The plug-and-play property also ensures that IFAD is readily applicable to a wide class of scientific models.

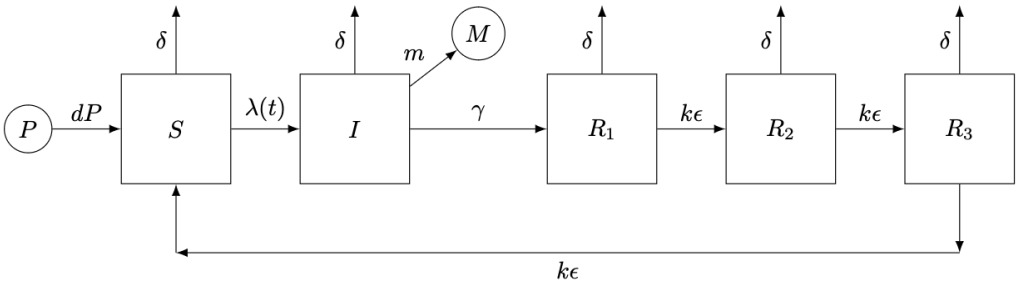


Fig. 2. A compartment flow diagram for the King et al. (2008) Dhaka cholera model.

We tested IFAD with $\alpha = 0$, $\alpha = 0.97$ and $\alpha = 1$ against IF2 on a global search problem for the Dhaka cholera model using our implementation in Pypomp 1.0.5 (Abkemeier et al., 2025). This example does not satisfy the conditions for our theoretical analysis of IFAD, so the successful empirical performance suggests these conditions are stronger than necessary. For each method, we performed 100 searches, initialized with 100 starting parameter vectors drawn uniformly from the same wide bounding box used by Ionides et al. (2015). To ensure a fair comparison under comparable computational effort, we ran each method for a similar amount of time. For each method, we chose algorithmic parameters that maximized the log-likelihood well for that method while finishing within the allotted time. We ran IF2 for 1950 iterations, while IFAD-0.97 is run using a short IF2 warm-start of 175 iterations followed by the Adam optimizer for 400 iterations. IFAD-0 and IFAD-1 were run for 60 IF2 iterations followed by 410 and 425 Adam iterations, respectively. The cooling rate used for IF2 was slower than that used for the warm start in IFAD. To assess whether other methods can catch up to IFAD-0.97 given more time, we separately ran the IF2-only approach for 4 times as many iterations and the IFAD variants with 4 times as many Adam iterations, slowing the cooling rate for each as well. A list of optimization settings is provided in the supplement (Section S9).

Our findings are summarized in Table 1 and Figures 3, 4, and 5. Because estimating the log-likelihoods at every iteration of IF2 with particle filtering for Figure 5 lengthens the computation time and thus compromises the fair comparison between methods, we obtained the times in Table 1 and Figure 5 by running the methods again without the extra calls to the particle filter. Every other value displayed for the Dhaka results comes from the runs with the extra particle filter calls.

Examining Table 1, IFAD-0.97 attained a log-likelihood of -3743.68, which surpassed IF2 even when IF2 ran for 4 times as long. IFAD-1 sometimes came close to IFAD-0.97, but its additional noise gave it worse median performance. IFAD-1 needed to be run for an extended time for the median log-likelihood to start catching up with IFAD-0.97.

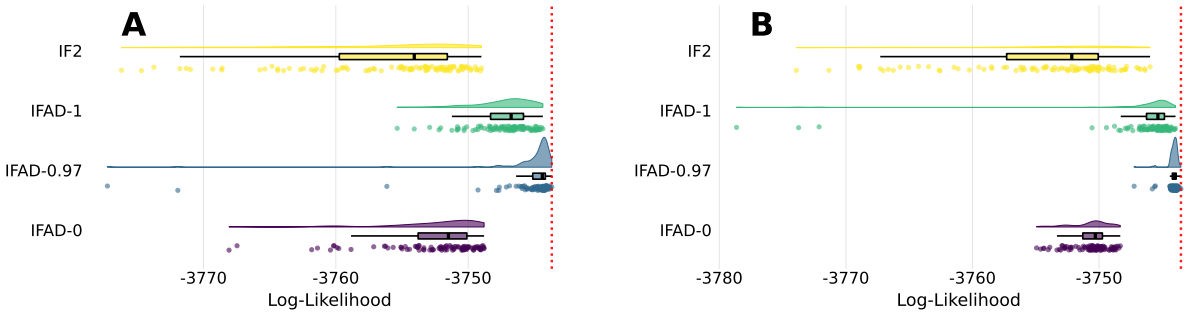


Fig. 3. Raincloud plot of the searches. *(a)* Comparable computational effort. *(b)* Extended computational effort. Log-likelihoods lower than -3800 are omitted. IFAD-0.97 outperforms all other methods. The dotted red line shows the best-known maximized log-likelihood.

IFAD-0 fell nearly 5 log-units short of IFAD-0.97 even with extended time, presumably due to its high bias. Based on Figure 5, IFAD-0 and IF2-only plateau heavily after being run for long enough and could not catch up to IFAD-0.97 within a reasonable timeframe.

Previously, a maximized log-likelihood of -3748.6 for this model and data was reported by King et al. (2016). This MLE was obtained with much computational effort, using many global and local IF2 searches and with the assistance of likelihood profiling, 8 years after the model was first proposed by King et al. (2008). Meanwhile, Ionides et al. (2015) only

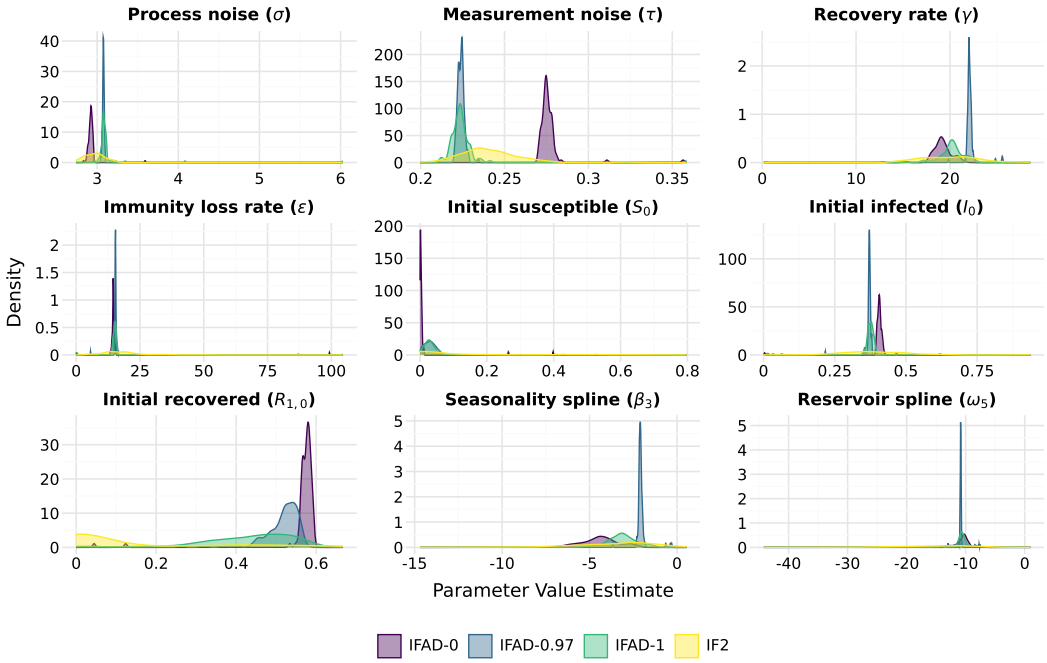


Fig. 4. Density plots of the final parameter estimates across the 100 replicates for nine key parameters of the cholera model under extended computational effort.

achieve a maximum log-likelihood of −3768.6, and King et al. (2008) only −3793.4. Using the improved computational efficiency of Pypomp (elaborated upon in Section 8), IF2 alone surpasses all of these previous results within a wall clock time of about an hour, while IFAD-0.97 does so in under 16 minutes and holds the record at −3743.53.

Notably, the log-likelihood and parameter estimates for IFAD-0.97 frequently exhibit much less Monte Carlo variance than that of the other methods, as can be seen in Figures 3 and 4. IFAD in general exhibits less Monte Carlo variance than IF2, especially for the parameter estimates. IFAD-0 has the second-lowest Monte Carlo variance for most parameter estimates, although it has the lowest for the initial state proportions S_0, I_0 and $R_{1,0}$. That said, the choice $\alpha = 0$ is also known to have the greatest bias, and IFAD-1 agrees more closely with IFAD-0.97. Because multiple searches are employed to mitigate the effects of Monte Carlo variance, the lower Monte Carlo variance of IFAD-0.97 suggests that fewer searches are needed when running IFAD-0.97 to obtain a log-likelihood

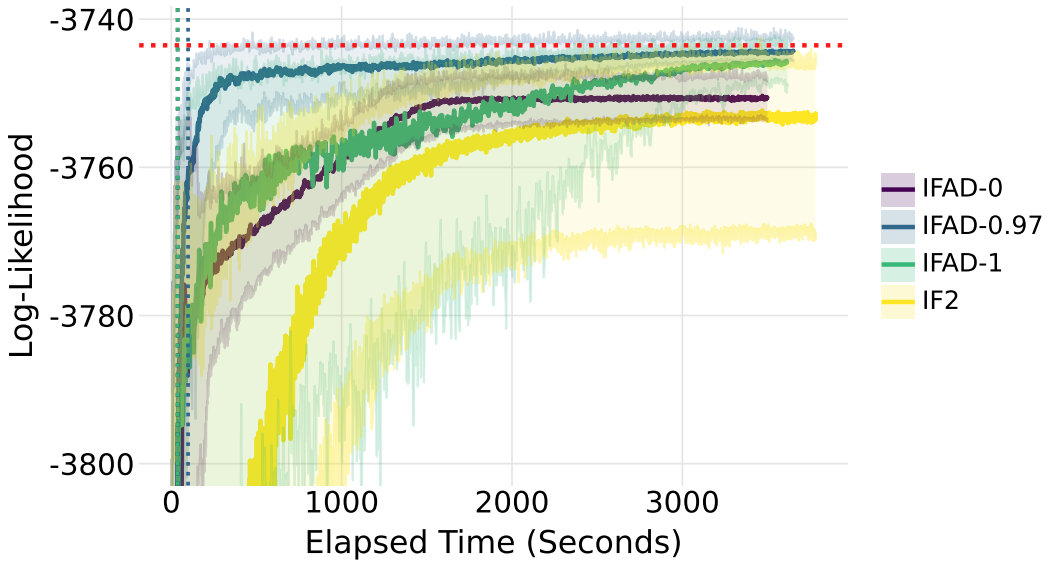


Fig. 5. Optimization progress of IFAD and IF2 under extended computational effort. The shaded regions cover the area between the 100th and 10th percentiles of log-likelihood estimates at each iteration. We use a dotted red line to display the MLE. The vertical dotted lines indicate where the IFAD runs switch from IF2 to Adam optimization. Some log-likelihood estimates appear to be slightly above the MLE; this is due to Monte Carlo evaluation error.

close to the maximum. Lowering the Monte Carlo variance also narrows the width of Monte-Carlo-adjusted profile confidence intervals (Ionides et al., 2017).

While IF2 effectively approaches a neighborhood of the MLE (Figure 5), IF2 alone ultimately fails to achieve the last few log-likelihood units, as IF2 struggles to get much closer than about 2 log-likelihood units of the MLE even when run for an extended time (Table 1). Conversely, we encountered many failed searches with gradient descent alone. This is a difficult, nonconvex, and noisy problem with 23 estimated parameters, and gradient descent without a warm-start gets stuck in local minima and saddle points.

IFAD, by comparison, approaches the MLE quickly due to the IF2 warm-start (Figure 5) and also succeeds at refining the coarse solution found by the warm-start with DMOP gradient steps to find the MLE. In this experiment, combining IF2 initialization

with DMOP refinement outperforms either method alone. Monte Carlo replication is appropriate for IFAD, or any Monte Carlo algorithm used to solve a challenging numerical problem, but Figure 5 shows that IFAD can find higher likelihood values more quickly and more reliably than the previous state-of-the-art. Since replicated searches are inevitably needed for Monte Carlo maximization of this challenging likelihood maximization task, it is important to give extra consideration to the right tail of the distributions in Figure 3 when comparing the capabilities of the methods.

6. Application to Bayesian Inference

Particle MCMC (Andrieu et al., 2010) is arguably the most popular method for full-information plug-and-play Bayesian inference for POMP models. However, particle Metropolis-Hastings often mixes slowly, requiring long burn-in periods. For instance, Fasiolo et al. (2016) ran 1.4 million iterations and discarded the first half as burn-in for a modified version of the King et al. (2008) model, using a different transition scheme, process noise and observation distribution. To alleviate this, we demonstrate a No-U-Turn Sampler (NUTS) (Hoffman and Gelman, 2014) using DMOP-α gradients, with a Gaussian prior common to NUTS and particle Metropolis–Hastings. The prior has independent Gaussian distributions on the estimation scale, including logarithms for the five positive parameters. Each mean is the midpoint of the corresponding parameter search interval, and each standard deviation is half its width, where we note that these intervals do not truncate the prior. Particle Metropolis–Hastings uses a Gaussian random walk with a full covariance matrix estimated from preliminary runs (Dahlin and Schön, 2019, p. 29), proposing fresh particle-filter random numbers at each iteration. This adapted proposal was crucial for convergence; we provide additional results in Supplementary Section S8 that depict slow mixing otherwise. NUTS uses DMOP-α gradients to construct trajectories conditional on the filter random numbers, which are updated separately by a Metropolis step. Its preconditioning matrix is also estimated from preliminary PMMH

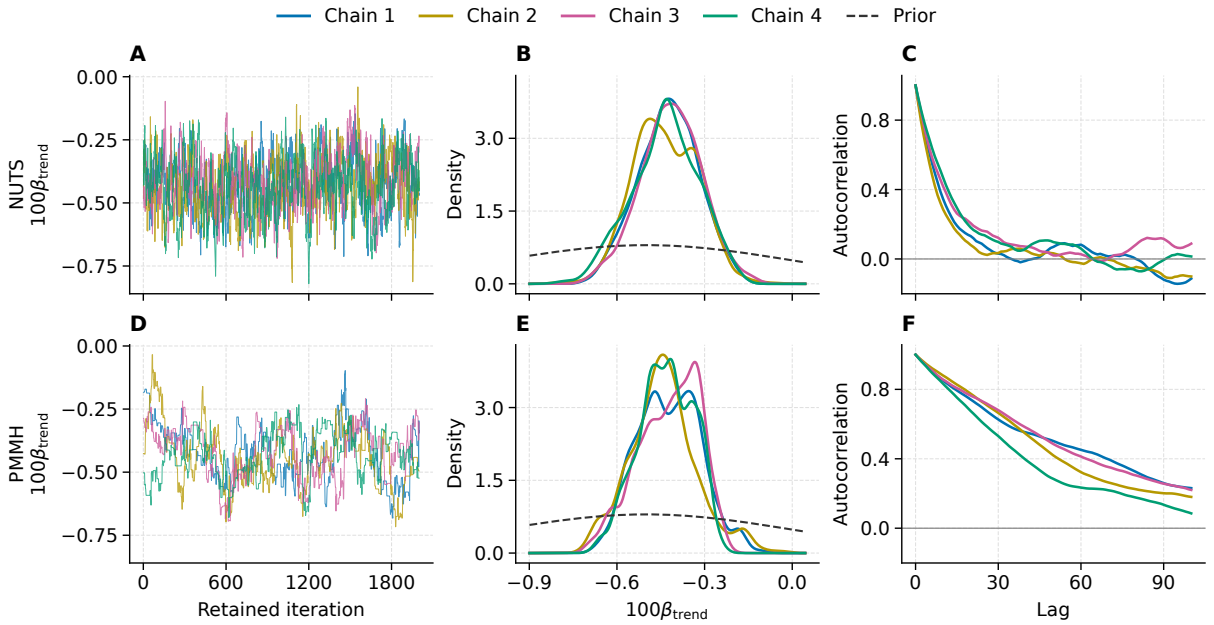


Fig. 6. Sampling diagnostics for the transformed trend coordinate $100\beta_{\text{trend}}$ in the Dhaka cholera model of King et al. (2008). Four chains of each method have 2,000 retained iterations after 500 warmup iterations. Panels show the traces, marginal density estimates, and autocorrelations of NUTS (A–C) NUTS and particle Metropolis–Hastings (D–F) respectively. The common prior is depicted in panels B and E with dashed lines.

runs. Both matrices are held fixed, with scalar step sizes adapted during the 500 warmup iterations. Supplementary Section S8 gives the prior parameters and proposal settings.

Using NUTS with DMOP-α gives promising preliminary results for parameter-space exploration after discarding 500 iterations (Figure 6). NUTS has been implemented with previous differentiable particle filters (Rosato et al., 2022) but not in our plug-and-play context with control over the bias/variance tradeoff.

With the same prior and starting points, the NUTS sampler explores the trend parameter more effectively than particle Metropolis–Hastings (Figure 6). We include quantitative comparisons for all estimated parameters in the SI. These preliminary results suggest that plug-and-play gradient estimates could assist particle Bayesian computation for complex scientific models.

7. Theoretical Analysis of MOP-α

Here, we show that MOP-α targets the filtering distribution and likelihood under θ, is consistent when $\alpha \uparrow 1$, and characterize rates for its bias and variance under different choices of α. To do so, we require the following assumptions:

(A1) **Continuity of the Likelihood.** $\ell(\theta)$ has more than two continuous derivatives in a neighborhood $\{\theta : \ell(\theta) > \lambda_1\}$ for some $\lambda_1 < \sup_\varphi \ell(\varphi)$.

(A2) **Bounded Process Model.** There exist $\underline{\mathrm{M}}, \bar{M}$ such that $0 < \underline{\mathrm{M}} \leq f_{X_n|X_{n-1}}(x_n|x_{n-1};\theta) \leq \bar{M} < \infty$.

(A3) **Bounded Measurement Model.** Uniformly in n, j, states, simulator draws, reference-filter resampling indices, and θ near ϕ, $0 < \underline{\mathrm{G}} \leq f_{Y_n|X_n}(y_n^* \mid x_n;\theta) \leq \bar{G} < \infty$, $|\partial_{\theta_a} \log g_{n,j}^{A,F,\theta}| \leq G'(\theta) < \infty$, and $|\partial^2_{\theta_a\theta_b} \log g_{n,j}^{A,F,\theta}| \leq G''(\theta) < \infty$ for $1 \leq a, b \leq p$. The derivatives include the dependence of the simulated state on θ. The symbols G' and G'' denote separate derivative bounds, not derivatives of the function G in (A4).

(A4) **Bounded Gradient Estimates.** There are functions $G(\theta), H(\theta) : \Theta \to [0, \infty)$ uniformly bounded by $G^*, H^* < \infty$, such that the MOP-α derivatives at $\theta = \phi$ satisfy $\|\nabla_\theta \hat{\ell}_J^\alpha(\theta)\|_2 \leq G(\theta)$ and $\|\nabla^2_\theta \hat{\ell}_J^\alpha(\theta)\|_{\mathrm{op}} \leq H(\theta)$ almost surely, uniformly in J and α. For $\alpha = 1$, these bounds hold on one probability-one event simultaneously for all J and all θ in a fixed closed ball K of positive radius centered at ϕ and contained in the interior of Θ. All estimates are coupled on one probability space, with the simulator draws and reference-filter resampling indices generated at ϕ held fixed as θ varies.

(A5) **Differentiable Density Ratios and Simulator.** For fixed ω, and any fixed collection of times, the simulated path and its log measurement densities have more than two continuous derivatives in θ.

(A6)**Measurement-sensitivity decay.** The simulator is C^1 in its incoming state. Starting the simulator at $X_m^\theta = x_m$, with θ and subsequent simulator draws held fixed,

$$\left\| \nabla_{x_m} \log f_{Y_n|X_n}\big(y_n^* \mid X_n^\theta; \theta\big) \right\|_2 \le C\kappa^{n-m}, \qquad C < \infty, \quad 0 < \kappa < 1.$$

The bound is uniform in $m \le n$, x_m, X_n^θ, ω, and θ near ϕ. The parameter derivatives of the initial state, simulator, and log measurement density are bounded, holding incoming states fixed.

(A6-J)**Alternative assumption to (A6).** The log measurement density is C^1 in (x, θ), and (A2) holds for θ near ϕ for the joint Markov process $(X_n^\theta, \partial_\theta X_n^\theta)$ relative to a common finite measure on its invariant state space.

Assumptions (A6) and (A6-J) provide two ways to control the simulated derivatives. Under (A6), the effect of an earlier state on a later measurement decays with the elapsed time, holding the simulator draws fixed. Under (A6-J), the state and its parameter derivative instead satisfy a joint mixing condition. Mixing of the physical state alone does not ensure either property. We require one of these additional assumptions for the particle error bounds; neither requires the algorithm to evaluate transition densities. Section S7 gives a sufficient condition for (A6) and an example satisfying (A6-J) but not (A6).

In particular, assumptions (A2) and (A3) imply that the POMP model satisfies a strong mixing condition, and as such are fairly strong but commonplace in the literature (Karjalainen et al., 2023; Del Moral, 2004). The fact that the likelihood estimate yielded by MOP-α has more than two continuous derivatives in θ follows from the construction of the likelihood estimate in equations ()1) and (2), as well as Assumptions (A3) and (A5).

We show that MOP-α targets the filtering distribution under θ and is strongly consistent for the likelihood under θ when $\alpha = 1$ or $\theta = \phi$. Employing $\alpha < 1$ lead to inconsistency when θ deviates from ϕ, but this bias disappears as θ approaches ϕ.

While the result is presented here as specific to MOP-α, we prove a more general result in the supplement (Section S4). This is a strong law of large numbers for triangular arrays

of particles resampled arbitrarily, with resampling probabilities not proportional to the targeted distribution of interest, but reweighted to correct for the cumulative discrepancy between the resampling and the target distribution.

Theorem 3 (MOP-α Targets the Filtering Distribution and Likelihood)**.** *When $\alpha = 1$ or $\theta = \phi$, MOP-α targets the filtering distribution under θ and is strongly consistent for the likelihood under θ. That is, for $\pi_n(\theta) = f_{X_n|Y_{1:n}}(x_n|y^*_{1:n};\theta)$ and any measurable and bounded functional h and for $\hat{\mathcal{L}}(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\prod_{n=1}^N L_n^{B,\theta,\alpha}$, it holds that*

$$\frac{\sum_{j=1}^J h(x_{n,j}^{F,\theta})\, w_{n,j}^{F,\theta}}{\sum_{j=1}^J w_{n,j}^{F,\theta}} \overset{a.s.}{\to} E_{\pi_n(\theta)}\big[h(X_n)\big], \quad \hat{\mathcal{L}}(\theta) \overset{a.s.}{\to} \mathcal{L}(\theta).$$

Proof. We provide a proof sketch here, deferring the details to the supplement (Section S4). When $\theta = \phi$, the ratio $g_{n,j}^{\theta}/g_{n,j}^{\phi} = 1$ regardless of α, reducing to the vanilla particle filter. When $\alpha = 1$ and $\theta \neq \phi$, suppose inductively that $\{(X_{n-1,j}^{F,\theta}, w_{n-1,j}^{F,\theta})\}_{j=1}^J$ targets $f_{X_{n-1}|Y_{1:n-1}}(x_{n-1}|y^*_{1:n-1};\theta)$. It can then be shown that $\{(X_{n,j}^{P,\theta}, w_{n,j}^{P,\theta})\}_{j=1}^J$ targets $f_{X_n|Y_{1:n-1}}(x_n|y^*_{1:n-1};\theta)$, that $\{(X_{n,j}^{P,\theta}, w_{n,j}^{P,\theta}\, g_{n,j}^{\theta})\}_{j=1}^J$ targets $f_{X_n|Y_{1:n}}(x_n|y^*_{1:n};\theta)$, and that weighting the particles by $(X_{n,j}^{F,\theta}, w_{n,j}^{F,\theta}) = (X_{n,k_j}^{P,\theta}, w_{n,k_j}^{P,\theta}\, g_{n,k_j}^{\theta}/g_{n,k_j}^{\phi})$, resampling the k_j with probabilities proportional to $g_{n,j}^{\phi}$, also targets $f_{X_n|Y_{1:n}}(x_n|y^*_{1:n};\theta)$. Assumption (A3) guarantees the boundedness and positivity conditions required by Lemma S7 for the measurement update. Estimating the likelihood with the before-resampling estimator $\hat{\mathcal{L}}(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$, we see that the strong consistency is a direct consequence of our result that $\{\big(X_{n,j}^{P,\theta}, w_{n,j}^{P,\theta}\big)\}$ targets $f_{X_n|Y_{1:n-1}}(x_n|y^*_{1:n-1};\theta)$. □

We take advantage of the equivalence between DMOP-α and the derivative of MOP-α to investigate the former by studying the latter. Specifically, we show the consistency of the MOP-1 gradient estimate below. Note that Poyiadjis et al. (2011) and Ścibior and Wood (2021) estimate the Fisher score by $\frac{1}{J}\sum_{j=1}^J \nabla_\theta \log f_{X_{0:N},Y_{1:N}}(x_{0:n,j}^{A,F,\theta}, y^*_{1:N};\theta)$

and not $\frac{1}{J}\sum_{j=1}^{J} \nabla_\theta \log f_{Y_{1:N}|X_{1:N}}(y^*_{1:N}|x^{A,F,\theta}_{1:n,j};\theta)$. The estimators are evidently somewhat similar, but it is not immediately apparent that they converge to the same thing.

Theorem 4 (Consistency of MOP-1 Gradient Estimate)**.** *The gradient estimate of MOP-α when $\alpha = 1$, $\theta = \phi$ is strongly consistent for the score: $\nabla_\theta \hat{\ell}^1_J(\theta) \overset{a.s.}{\to} \nabla_\theta \ell(\theta)$ as $J \to \infty$.*

We present a proof outline here, postponing the full argument to the supplement (Section S5). On the compact neighborhood K in Assumption (A4), Theorem 3 with $\alpha = 1$ gives pointwise almost-sure convergence of $\hat{\ell}^1_J(\theta)$ to $\ell(\theta)$, including when $\theta \neq \phi$. Intersecting these events over a countable dense subset of K, the gradient bound upgrades this to uniform convergence on K. The Hessian bound makes the gradients uniformly equicontinuous, so Arzelà–Ascoli gives uniformly convergent gradient subsequences. Passing to the limit in the fundamental theorem of calculus identifies each such limit as $\nabla_\theta \ell(\theta)$ in the interior of K. Thus the full gradient sequence converges locally uniformly there almost surely, and in particular $\nabla_\theta \hat{\ell}^1_J(\phi) \to \nabla_\theta \ell(\phi)$.

We now bound the particle approximation error and population discount bias of DMOP-α. The result describes the bias-variance tradeoff as α varies between zero and one.

Theorem 5 (Error bounds for DMOP-α)**.** *Suppose (A2), (A3), (A5), and either (A6) or (A6-J) hold. Fix the observations, use multinomial resampling, and let $J \geq 2$ and $\theta = \phi$. Write $H_N(\alpha) = \sum_{r=0}^{N-1} \alpha^r$, using the after-resampling likelihood estimate. For $0 \leq \alpha < 1$,*

$$\mathbb{E}\|\nabla_\theta \hat{\ell}^\alpha_J(\theta) - \nabla_\theta \ell(\theta)\|_2^2 \lesssim pG'(\theta)^2 \left[\left\{\frac{N(1+\log J)^2}{J} + \frac{N^2}{J^2}\right\} H_N(\alpha)^2 + N^2(1-\alpha)^2\right]. \quad (3)$$

At $\alpha = 1$,

$$\mathbb{E}\|\nabla_\theta \hat{\ell}^1_J(\theta) - \nabla_\theta \ell(\theta)\|_2^2 \lesssim pG'(\theta)^2 \min\{N^2, N^3/J\}. \quad (4)$$

The constants depend on the model and the stated stability bounds, independently of N, J, α*. Expectations are over the particle-filter randomness.*

The bound separates particle approximation error from population discount bias. The first term in Equation 3 controls the error relative to the population DMOP-α score, while the second controls its squared difference from the full score. Increasing the number of particles reduces the particle error, but note that it does not remove the population bias introduced by discounting.

Table 2 summarizes the bounds. We suppress the common factor $pG'(\theta)^2$, write $L_J = 1 + \log J$, and take $J \geq N$. The population bias is the bias remaining as $J \to \infty$.

For DMOP-0, the particle error grows at most linearly with the number of observations, apart from logarithmic factors. The joint mixing assumption (A6-J) gives one fewer factor of $1+\log J$ in this case. However, DMOP-0 can retain population bias even with arbitrarily many particles.

For any fixed $0 < \alpha < 1$, the particle bound has the same dependence on N and J, with a constant that increases as α approaches one. The population bias bound decreases at the same time. The exact memory factor is $H_N(\alpha) \leq \min\{N, (1-\alpha)^{-1}\}$, so the finite-horizon particle bound remains finite as α approaches one.

For DMOP-1, any bias introduced by population discounting is zero, and so the MSE tends to zero as J increases for fixed N. Its N^3/J upper bound allows substantially greater particle error for long series.

We analyze geometrically discounted scores through an exact weighted average of scores truncated at finite lags. Posterior forgetting controls the population discount bias, while particle approximation and covariance bounds control the Monte Carlo error.

The main difficulties are that measurement gradients depend on simulated histories and that resampling creates dependence between particles and across time. Under (A6), we expand each measurement gradient according to the age of its simulator contributions. Their geometric decay permits us to sum bounds for finite histories using mixing of the

physical particle cloud. Under (A6-J), the measurement gradient is already a bounded function of the current state–derivative pair, so we apply the particle bounds directly to that joint process. Section S6 develops the covariance argument for DMOP-0; Section S7 handles general discounting and proves the theorem.

8. Computational Efficiency

The cheap gradient principle of Kakade and Lee (2018) suggests that a gradient estimate from DMOP-α should take no more than 6 times that of the runtime of the particle filter. We found the empirical value of this factor to be 3.75, using the implementation of our methods in the Pypomp library (Abkemeier et al., 2025). DMOP-α and IFAD therefore share the same $O(NJ)$ time complexity as the particle filter, unlike the $O(NJ^2)$ complexity of Corenflos et al. (2021) and Algorithm 2 in (Poyiadjis et al., 2011; Ścibior and Wood, 2021).

Implementation of the particle filter, IF2, and DMOP-α in JAX (Bradbury et al., 2018) takes advantage of just-in-time compilation and GPU acceleration. While CPU-based implementations of the particle filtering methods use multiple CPU cores by parallelizing over independent replications of a given method, Pypomp uses tens of thousands of GPU cores by vectorizing over both particles and independent method replications. To examine the performance improvement on the Dhaka model, we ran 1 instance of IF2 with 60 iterations and 5000 particles using 1 CPU core and compared this with running 100 replications on a GPU. The results in Table 3 show that, despite running 100 times as many replications, the GPU finishes 26.0 times faster, giving it a throughput 2597.2 times greater than the single CPU core.

9. Discussion

Practical likelihood-based data analysis involves many likelihood optimizations (King et al., 2008; Blake et al., 2014; Pons-Salort and Grassly, 2018; Subramanian et al., 2021; Fox et al., 2022; Drake et al., 2023). This occurs with profile likelihood

confidence intervals and model selection. A careful scientist should consider many model variations, to see whether the conclusions of the study are sensitive to alternative model specifications. Coding model variations is relatively simple when using plug-and-play inference methodology. However, assessing the scientific potential of these variations requires likelihood optimization. Monte Carlo adjusted profile methods can help to acommodate maximization and evaluation error (Ionides et al., 2017; Ning et al., 2021) but these methods are most effective when the maximized log-likelihood can be obtained within a few units. Improving the process of model fitting has practical consequences for the ability of scientists to evaluate hypotheses. We have demonstrated, for the first time, a plug-and-play maximum likelihood approach which provides a high-quality log-likelihood maximization in the complex benchmark model of King et al. (2008). The *Daphnia* application in the supplement (Section S10) demonstrates that these gains also extend to complex panel data.

If the simulator is discontinuous in θ, DMOP-α no longer applies. We are working on a variation of DMOP-α applicable to this case. Specifically, differentiable transition density ratios can be used instead of a differentiable simulator. The off-parameter treatment of the measurement model is extended to an off-parameter simulation for the dynamic process. In that case, one requires access to the transition densities, or at least their ratios. This algorithm may be better suited to large discrete state spaces than either DMOP-α or differentiation of the Baum-Welch algorithm.

Discounting the weights by some $\alpha \in [0, 1]$ is not the only way to interpolate between the estimators of Naesseth et al. (2018) and Poyiadjis et al. (2011). As the proof of Theorem 5 implies, we can also truncate the weights at a fixed lag, corresponding to the MOP-$(1, k)$ algorithm mentioned. The analysis is similar, with comparable rates but with a slightly less convenient implementation.

Reproducibility

Source code for this article is archived on Zenodo with DOI 10.5281/zenodo.23538801. It is also available on GitHub at https://github.com/pypomp/dmop.

Acknowledgments

This research was supported by National Science Foundation grants DMS-1761603, DEB-1933497 and OAC-2607869. We thank Nicolas Chopin for communications regarding the strong law of large numbers for off-parameter resampled particle filters, and other feedback. We thank Jun Chen and Arnaud Doucet for helpful discussions. Feedback from two anonymous referees, an associate editor, and generative artificial intelligence (AI) has led to improvements in this revision. AI was also employed to support the coding of numerical results. We used frontier AI models from OpenAI and Anthropic, up to and including Claude Opus 5.5 and GPT-6 Astra.

References

Abkemeier, A. J., Chen, J., Tan, K., Wheeler, J., He, K., Yang, B., King, A. A., Terhorst, J., Hooker, G., and Ionides, E. L. (2025). Pypomp: Inference for partially observed Markov processes with Python and JAX. http://github.com/pypomp.

Andrieu, C., Doucet, A., and Holenstein, R. (2010). Particle Markov chain Monte Carlo methods. *Journal of the Royal Statistical Society, Series B*, 72:269–342.

Arulampalam, M. S., Maskell, S., Gordon, N., and Clapp, T. (2002). A tutorial on particle filters for online nonlinear, non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50:174 – 188.

Arya, G., Schauer, M., Schäfer, F., and Rackauckas, C. (2022). Automatic differentiation of programs with discrete randomness. *Advances in Neural Information Processing Systems*, 35:10435–10447.

Blake, I. M., Martin, R., Goel, A., Khetsuriani, N., Everts, J., Wolff, C., Wassilak, S., Aylward, R. B., and Grassly, N. C. (2014). The role of older children and adults

in wild poliovirus transmission. *Proceedings of the National Academy of Sciences*, 111(29):10604–10609.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. (2018). JAX: Composable transformations of Python+NumPy programs. http://github.com/google/jax.

Bretó, C. (2014). On idiosyncratic stochasticity of financial leverage effects. *Statistics and Probability Letters*, 91:20–26.

Chen, X. and Li, Y. (2024). Normalizing flow-based differentiable particle filters. *IEEE Transactions on Signal Processing*, 73:493–507.

Corenflos, A., Thornton, J., Deligiannidis, G., and Doucet, A. (2021). Differentiable particle filtering via entropy-regularized optimal transport. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 2100–2111. PMLR.

Dahlin, J. and Schön, T. B. (2019). Getting started with particle Metropolis–Hastings for inference in nonlinear dynamical models. *Journal of Statistical Software*, 88(2):1–41.

Del Moral, P. (2004). *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, New York.

Ditlevsen, S. and Samson, A. (2019). Hypoelliptic diffusions: Filtering and inference from complete and partial observations. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 81(2):361–384.

Doucet, A., Jacob, P. E., and Rubenthaler, S. (2013). Derivative-free estimation of the score vector and observed information matrix with application to state-space models. *arXiv:1304.5768v3*.

Drake, J. M., Handel, A., Marty, É., O'Dea, E. B., O'Sullivan, T., Righi, G., and Tredennick, A. T. (2023). A data-driven semi-parametric model of SARS-CoV-2 transmission in the United States. *PLOS Computational Biology*, 19(11):e1011610.

Fasiolo, M., Pya, N., and Wood, S. N. (2016). A comparison of inferential methods for highly nonlinear state space models in ecology and epidemiology. *Statistical Science*, 31(1):96–118.

Fox, S. J., Lachmann, M., Tec, M., Pasco, R., Woody, S., Du, Z., Wang, X., Ingle, T. A., Javan, E., Dahan, M., Gaither, K., Escott, M. E., Adler, S. I., Johnston, S. C., Scott, J. G., and Meyers, L. A. (2022). Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119.

Griewank, A. and Walther, A. (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM.

He, D., Ionides, E. L., and King, A. A. (2010). Plug-and-play inference for disease dynamics: Measles in large and small towns as a case study. *Journal of the Royal Society Interface*, 7:271–283.

Hoffman, M. D. and Gelman, A. (2014). The No-U-turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(1):1593–1623.

Ionides, E. L., Bretó, C., and King, A. A. (2006). Inference for nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 103:18438–18443.

Ionides, E. L., Breto, C., Park, J., Smith, R. A., and King, A. A. (2017). Monte Carlo profile confidence intervals for dynamic systems. *Journal of the Royal Society Interface*, 14:1–10.

Ionides, E. L., Nguyen, D., Atchadé, Y., Stoev, S., and King, A. A. (2015). Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719—-724.

Jonschkowski, R., Rastogi, D., and Brock, O. (2018). Differentiable particle filters: End-to-end learning with algorithmic priors. *Robotics: Science and Systems XIV*.

Kakade, S. M. and Lee, J. D. (2018). Provably correct automatic subdifferentiation for qualified programs. *Advances in Neural Information Processing Systems*, 31.

Karjalainen, J., Lee, A., Singh, S. S., and Vihola, M. (2023). On the forgetting of particle filters. *arXiv:2309.08517*.

Karppinen, S., Singh, S. S., and Vihola, M. (2024). Conditional particle filters with bridge backward sampling. *Journal of Computational and Graphical Statistics*, 33(2):364–378.

Kim, J. and Stoffer, D. S. (2008). Fitting stochastic volatility models in the presence of irregular sampling via particle methods and the EM algorithm. *Journal of Time Series Analysis*, 29(5):811–833.

King, A. A., Ionides, E. L., Pascual, M., and Bouma, M. J. (2008). Inapparent infections and cholera dynamics. *Nature*, 454:877–880.

King, A. A., Nguyen, D., and Ionides, E. L. (2016). Statistical inference for partially observed Markov processes via the R package pomp. *Journal of Statistical Software*, 69:1–43.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.

Knape, J. and de Valpine, P. (2012). Fitting complex population models by combining particle filters with Markov chain Monte Carlo. *Ecology*, 93(2):256–263.

Le Cam, L. and Yang, G. L. (2000). *Asymptotics in Statistics*. Springer, New York, 2nd edition.

Liu, J. and West, M. (2001). Combining parameter and state estimation in simulation-based filtering. In Doucet, A., de Freitas, N., and Gordon, N., editors, *Sequential Monte Carlo Methods in Practice*, pages 197–224. Springer, New York.

Malik, S. and Pitt, M. K. (2011). Particle filters for continuous likelihood evaluation and maximisation. *Journal of Econometrics*, 165(2):190–209.

Naesseth, C., Linderman, S., Ranganath, R., and Blei, D. (2018). Variational sequential Monte Carlo. In Storkey, A. and Perez-Cruz, F., editors, *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, volume 84 of *Proceedings of Machine Learning Research*, pages 968–977. PMLR.

Ning, N., Ionides, E. L., and Ritov, Y. (2021). Scalable Monte Carlo inference and rescaled local asymptotic normality. *Bernoulli*, 27:2532–2555.

Owen, J., Wilkinson, D. J., and Gillespie, C. S. (2015). Likelihood free inference for Markov processes: A comparison. *Statistical Applications in Genetics and Molecular Biology*, 14(2):189–209.

Pons-Salort, M. and Grassly, N. C. (2018). Serotype-specific immunity explains the incidence of diseases caused by human enteroviruses. *Science*, 361(6404):800–803.

Poyiadjis, G., Doucet, A., and Singh, S. S. (2011). Particle approximations of the score and observed information matrix in state space models with application to parameter estimation. *Biometrika*, 98(1):65–80.

Roberts, G. O. and Stramer, O. (2001). On inference for partially observed nonlinear diffusion models using the Metropolis-Hastings algorithm. *Biometrika*, 88:603–621.

Roosta-Khorasani, F. and Mahoney, M. W. (2016). Sub-sampled Newton methods I: Globally convergent algorithms. *arXiv:1601.04737*.

Rosato, C., Devlin, L., Beraud, V., Horridge, P., Schön, T. B., and Maskell, S. (2022). Efficient learning of the parameters of non-linear models using differentiable resampling in particle filters. *IEEE Transactions on Signal Processing*, 70:3676–3692.

Ścibior, A. and Wood, F. (2021). Differentiable particle filtering without modifying the forward pass. *arXiv:2106.10314*.

Singh, A., Makhlouf, O., Igl, M., Messias, J., Doucet, A., and Whiteson, S. (2023). Particle-based score estimation for state space model learning in autonomous driving. *Conference on Robot Learning*, pages 1168–1177.

Stocks, T., Britton, T., and Höhle, M. (2018). Model selection and parameter estimation for dynamic epidemic models via iterated filtering: Application to rotavirus in Germany. *Biostatistics*, 21(3):400–416.

Subramanian, R., He, Q., and Pascual, M. (2021). Quantifying asymptomatic infection and transmission of COVID-19 in New York City using observed cases, serology, and testing capacity. *Proceedings of the National Academy of Sciences*, 118(9).

Svensson, A., Lindsten, F., and Schön, T. B. (2018). Learning nonlinear state-space models using smooth particle-filter-based likelihood approximations. *IFAC–PapersOnLine*, 51(15):652–657.

Toni, T., Welch, D., Strelkowa, N., Ipsen, A., and Stumpf, M. P. (2009). Approximate Bayesian computation scheme for parameter inference and model selection in dynamical systems. *Journal of the Royal Society Interface*, 6:187–202.

Wood, S. N. (2010). Statistical inference for noisy nonlinear ecological dynamic systems. *Nature*, 466:1102–1104.

Wycoff, N., Smith, J. W., Booth, A. S., and Gramacy, R. B. (2026). Voronoi candidates for Bayesian optimization. *Journal of Global Optimization*, 94:175–201.

Yang, B., Wheeler, J., Duffy, M. A., King, A. A., and Ionides, E. L. (2025). Mechanistic models for panel data: Analysis of ecological experiments with four interacting species. *arXiv:2506.04508*.

	Maximum	Re-evaluated	Median	IF2 time (s)	DMOP time (s)
Comparable computational effort (≈ 15 minutes)					
IFAD-0.97	-3743.68	-3744.26	-3744.40	92	851
IFAD-1	-3744.36	-3744.40	-3746.84	35	902
IFAD-0	-3748.80	-3749.02	-3751.62	36	912
IF2	-3749.00	-3748.97	-3754.24	946	-
Extended computational effort (≈ 60 minutes)					
IFAD-0.97	-3743.53	-3744.30	-3744.03	96	3557
IFAD-1	-3743.96	-3744.33	-3745.39	35	3583
IF2	-3745.97	-3746.11	-3752.63	3782	-
IFAD-0	-3748.32	-3748.38	-3750.37	35	3465

Table 1. Maximum log-likelihood found by IF2 and IFAD under both comparable and extended computational effort. The maximum is the largest of the 100 searches, and is biased upward by the winner's curse; the re-evaluated column gives a fresh particle filter evaluation of that same search, which removes the selection bias at the cost of being a single noisy evaluation. Where the searches cluster tightly, as they do for IFAD-0.97 under extended effort, the re-evaluation regresses toward the median. IFAD-0.97 performs the best among all methods.

	Particle MSE		Squared population
Choice of α	Under (A6)	Under (A6-J)	bias
$\alpha = 0$	N/J	N/J	N^2
Fixed $0 < \alpha < 1$	$\frac{NL_J^2}{J(1-\alpha)^2}$	$\frac{NL_J}{J(1-\alpha)^2}$	$N^2(1-\alpha)^2$
$\alpha = 1$	N^3/J	N^3/J	0

Table 2. Upper bounds for DMOP-α with $J \geq N$, omitting the common factor $pG'(\theta)^2$, model-dependent constants, and logarithmic factors. The particle MSE bounds also bound the variance; adding the population term bounds the full-score MSE up to a constant.

Metric	`pomp` (King et al., 2016)	Pypomp (Abkemeier et al., 2025)
Language	R (C simulator)	Python (JAX)
Hardware	3.0 GHz Intel Xeon Gold 6154 (1 core)	NVIDIA RTX6000 Pro Blackwell
IF2 Replications	1	100
Latency (s)	935	36

Table 3. Comparison of execution latency for running IF2 (60 iterations, 5000 particles, 1 versus 100 replications) between the `pomp` (CPU-only, R and compiled C) and Pypomp (GPU-accelerated, compiled Python).